



CSable

Système de Contrôle pour Robot Quadrupède à Mécanisme à Liaisons Parallèles

Documentation Technique Officielle — Guide de Référence & Manuel d'Utilisation

Version	1.0.0
Date	Juin 2026
Référence	CSable-DOC-001
Domaine	Robotique — Locomotion Legged
Plateforme	MATLAB R2023b+ Dynamixel SDK
Langue	Français

Table des Matières

1. Introduction et Présentation du Projet

- 1.1 Contexte et Objectifs
- 1.2 Architecture Logicielle

2. Environnement et Prérequis

- 2.1 Prérequis Logiciels
- 2.2 Matériel Requis
- 2.3 Installation et Configuration

3. Module de Cinématique

- 3.1 Mécanisme à Liaisons Parallèles — Description
- 3.2 Cinématique Directe (forward_kinematics.m)
- 3.3 Cinématique Inverse (inverse_kinematics.m)
- 3.4 Calcul du Jacobien (compute_jacobian.m)
- 3.5 Analyse de l'Espace de Travail (calc_workspace.m)

4. Module de Contrôle

- 4.1 Classe de Base Abstraite (AbstractController.m)
- 4.2 Contrôle en Boucle Ouverte (OpenLoopControl.m)
- 4.3 Contrôle par Modèle Virtuel (VirtualModelControl.m)
- 4.4 Contrôle Prédictif par Modèle — MPC (Réservé)

5. Interface Matérielle

- 5.1 DynamixelInterface.m — Contrôle de Position
- 5.2 DynamixelInterfaceVMC.m — Contrôle en Couple
- 5.3 Utilitaires de Conversion (rad2dxl / dxl2rad)

6. Scripts Principaux d'Exécution

- 6.1 Démarche en quadrupède — main_hardware_standard_gait.m
- 6.2 Trajectoire cycloïde — main_hardware_cycloid.m
- 6.3 Trajectoire elliptique — main_hardware_ellipse.m
- 6.4 Démarche lente — main_hardware_slow_gait.m

7. Tests et Visualisation

- 7.1 Visualiseur Interactif (leg_viewer.m)
- 7.2 Analyse de l'Espace de Travail (test_workspace.m)
- 7.3 Trajectoire Biologique (main_cycloid_fennec.m)

8. Paramètres de Configuration Centraux

9. Bonnes Pratiques et Sécurité

10. Glossaire

1 Introduction et Présentation du Projet

1.1 Contexte et Objectifs

CSable est un système de contrôle temps réel développé sous MATLAB pour un robot quadrupède à mécanisme à liaisons parallèles. Le robot utilise des servomoteurs **Dynamixel XM430-W350-R** et des moteurs auxiliaires **AX-12** pour assurer la stabilité latérale (axe Z). L'objectif principal est de fournir une architecture logicielle modulaire, extensible et sûre, capable de piloter les quatre pattes du robot selon différents schémas de démarche tout en garantissant des contraintes temps réel à 100 Hz.

Le projet est organisé selon les conventions de packages MATLAB (**+namespace**), ce qui garantit l'isolation des responsabilités entre les modules de cinématique, de contrôle, d'interface matérielle et de configuration.

Domaine	Robotique mobile — Locomotion quadrupède — Mécanisme à quatre barres parallèles — Commande temps réel — Servomoteurs Dynamixel Protocol 2.0.
---------	--

1.2 Architecture Logicielle

Le projet suit une architecture orientée objet (OOP) avec séparation claire entre la couche matérielle, la couche cinématique et la couche de contrôle :

Dossier/Package	Rôle	Fichiers clés
config/	Paramètres centraux du robot	robot_params.m
+core/	Classes de base et interfaces	AbstractController.m, Robot.m
+control/	Algorithmes de contrôle	OpenLoopControl.m, VirtualModelControl.m, ModelPredictiveControl.m
+hardware/	Communication moteurs	DynamixelInterface.m, DynamixelInterfaceVMC.m
+kinematics/	Résolution cinématique	forward_kinematics.m, inverse_kinematics.m, compute_jacobian.m, calc_workspace.m
+utils/	Conversions d'unités	rad2dxl.m, dxl2rad.m
tests/	Visualisation et validation	leg_viewer.m, test_workspace.m, main_cycloid_fennec.m
(racine)/	Scripts d'exécution principaux	main_hardware_*.m

2 Environnement et Prérequis

2.1 Prérequis Logiciels

Logiciel	Version minimale	Utilisation
MATLAB	R2023b	Environnement d'exécution principal
Dynamixel SDK (MATLAB)	≥ 3.7	Communication Protocol 2.0 et 1.0
Control System Toolbox	Recommandée	Visualisation et analyse de systèmes

2.2 Matériel Requis

Composant	Quantité	Spécification
Servomoteur XM430-W350-R	8 (min.)	Protocol 2.0 — 1 Mbaud — 12V
Moteur AX-12A	4	Protocol 1.0 — Axe latéral Z
Interface U2D2	1	USB $\leftrightarrow$ TTL RS-485
Alimentation 12V	1	$\geq 5A$ recommandé
PC Windows/Linux	1	Port USB disponible

2.3 Installation et Configuration

Étape 1 — Dynamixel SDK : Télécharger et installer le SDK depuis le dépôt officiel ROBOTIS. Ajouter le chemin au path MATLAB :

```
addpath(genpath('C:/DynamixelSDK/matlab'));  
savepath;
```

Étape 2 — Port série : Identifier le port COM de l'interface U2D2 (ex. COM5 sous Windows) et le renseigner dans `config/robot_params.m` :

```
params.PORT_NAME = 'COM5'; % Adapter selon le système  
params.BAUD_RATE = 1000000;
```

Étape 3 — Lancement du projet : Ouvrir MATLAB dans le répertoire racine de CSable (le dossier contenant les packages +core, +control, etc.) afin que les namespaces soient disponibles.

- Ne jamais lancer un script principal sans vérifier préalablement que les moteurs sont alimentés et mécaniquement en sécurité. Une erreur de cinématique inverse peut provoquer des positions singulières.

3 Module de Cinématique

3.1 Mécanisme à Liaisons Parallèles — Description

Chaque patte du robot CSable est un mécanisme à liaisons parallèles à deux degrés de liberté. Ce mécanisme est composé de deux boucles cinématiques fermées :

- **Boucle supérieure (quadrilatère à quatre barres)** : A–B–C–D, où A est le point fixe, B est la manivelle (crank AB), C est la bielle (coupler), D est l'articulation de la hanche (hip joint).
- **Triangle rigide** : C–D–E forment une plaque triangulaire rigide liée à la bielle.
- **Boucle inférieure (genou)** : D–E–F–G, où G est l'articulation du genou et EF est la bielle de tirage (pull-rod).
- **Tibia** : G–H, le pied H est l'effecteur final dans l'espace cartésien.

Segment	Longueur (mm)	Description
AD	41.00	Écartement moteurs (fixe)
AB	20.10	Manivelle (crank)
BC	29.49	Bielle supérieure (coupler)
CD	28.07	Côté supérieur du triangle rigide
DE	27.94	Côté inférieur du triangle rigide
CE	38.18	Diagonale du triangle rigide
EF	100.00	Bielle de tirage (pull-rod)
FG	27.27	Manivelle du genou
DG	100.00	Cuisse (thigh, L2)
GH	105.73	Tibia (shin, L3)

Le repère de référence est centré sur D (hanche). L'axe X représente la direction avant/arrière, et l'axe Y l'axe vertical (négatif vers le sol). La position nominale du pied en appui est typiquement $Y \approx -150$ mm.

3.2 Cinématique Directe — forward_kinematics.m

Cette fonction calcule la position cartésienne du pied H à partir des angles moteurs $[\theta_1, a]$. Elle résout les deux boucles cinématiques par la loi des cosinus et les intersections cercle-cercle.

```
% Signature
[pos, points] = forward_kinematics(q, cfg)
%
```

```
% Entrées :
% q — [theta2; a] angles moteurs (rad)
% cfg — structure de configuration (longueurs des segments)
%
% Sorties :
% pos — [X; Y] position du pied (mm)
% points — structure A..H pour visualisation
```

La configuration d'assemblage est définie par trois paramètres :

- **branch_knee** = **-1** : flexion du genou dans le sens mammifère (chien).
- **branch_E** = **-1** : sélection du côté de l'intersection pour la boucle inférieure.
- **branch_crank** = **+1** : la manivelle se plie vers le haut.

3.3 Cinématique Inverse — `inverse_kinematics.m`

La cinématique inverse résout les angles moteurs $[\theta, a]$ à partir de la position souhaitée du pied $[X, Y]$. Elle suit la séquence inverse de la cinématique directe :

```
% Signature
[q, valid] = inverse_kinematics(pos, cfg)
%
% Entrées :
% pos — [X; Y] position cible du pied (mm)
% cfg — structure de configuration
%
% Sorties :
% q — [theta2; a] (rad) — [NaN; NaN] si hors espace de travail
% valid — booléen indiquant la réussite
```

■ La fonction retourne $[\text{NaN}; \text{NaN}]$ si la position cible est hors de l'espace de travail atteignable. Le contrôleur doit impérativement vérifier cette valeur avant d'envoyer une commande aux moteurs.

3.4 Calcul du Jacobien — `compute_jacobian.m`

Le Jacobien numérique est calculé par différences finies centrées ($\delta = 10^{-6}$ rad). Il est utilisé par le contrôle par modèle virtuel pour transformer les forces cartésiennes en couples moteurs : $\tau = J^T \cdot F$.

```
% Signature
J = compute_jacobian(q, cfg)
%
% Retourne une matrice 2x2 : [dx/dθ, dx/da ; dy/dθ, dy/da]
% Retourne des colonnes nulles si singularité détectée.
```

3.5 Analyse de l'Espace de Travail — `calc_workspace.m`

Cette fonction balaie l'espace des configurations articulaires sur une grille 60×60 (θ : 90° – 180° , a : 95° – 175°), appelle la cinématique directe pour chaque point, filtre les résultats invalides (NaN) et retourne le nuage de points atteignables. Elle est utilisée pour valider la trajectoire prévue avant l'exécution.


```
[X, Y] = calc_workspace(cfg)
% Visualise le nuage de points + contour convexe.
```

4 Module de Contrôle

4.1 Classe de Base Abstraite — AbstractController.m

Tous les contrôleurs héritent de **core.AbstractController** et doivent implémenter la méthode abstraite `computeAction(ref)`. La classe définit également la période d'échantillonnage $T_s = 0.01$ s (100 Hz) par défaut.

```
classdef AbstractController < handle
    properties
        Ts = 0.01 % Période d'échantillonnage (s)
    end
    methods (Abstract)
        action = computeAction(obj, ref) % Doit être surchargée
    end
end
```

4.2 Contrôle en Boucle Ouverte — OpenLoopControl.m

Le contrôleur en boucle ouverte (**control.OpenLoopControl**) reçoit une position cartésienne cible $[X ; Z]$, applique la cinématique inverse et retourne les angles moteurs. Il intègre :

- **Limitation de vitesse** : 240°/s maximum entre deux pas.
- **Détection de singularité** : conservation de la dernière action valide si IK retourne NaN.
- **Mémoire de l'action précédente** (PreviousAction) pour le lissage.

```
ctrl = control.OpenLoopControl(robot_params);
action = ctrl.computeAction([X; Y]);
% action = [theta2; a] en radians
```

Paramètre	Valeur par défaut	Unité	Description
MAX_VEL	240	°/s	Limite de vitesse angulaire
Ts	0.01	s	Période d'échantillonnage
PreviousAction	[NaN; NaN]	rad	Warm-start — initialisé au 1er appel

4.3 Contrôle par Modèle Virtuel — VirtualModelControl.m

Le contrôle par modèle virtuel (VMC) simule des ressorts et amortisseurs virtuels entre la position souhaitée et la position réelle du pied. La force virtuelle calculée est ensuite transformée en couples moteurs via le Jacobien transposé.

Loi de commande : $F = K_p \cdot e + K_d \cdot \dot{e} \rightarrow \tau = J^T \cdot F$

Paramètre	Valeur	Unité	Description
Kp_x	500	N/m	Raideur virtuelle — axe X
Kp_z	300	N/m	Raideur virtuelle — axe Z
Kd	10	N·s/m	Amortissement virtuel
τ_max	±3.0	N·m	Saturation couple (moteur : 4.1 N·m max)

4.4 Contrôle Prédictif par Modèle (MPC) — Réservé

Le fichier **ModelPredictiveControl.m** constitue un squelette d'implémentation du MPC avec matrices de pondération Q/R et horizon de prédiction configurable. Dans la version actuelle, computeAction() retourne zéro — l'implémentation complète est réservée aux travaux futurs.

5 Interface Matérielle

5.1 DynamixelInterface.m — Contrôle de Position

La classe `hardware.DynamixelInterface` gère la communication bidirectionnelle avec les servomoteurs Dynamixel XM430 (Protocol 2.0) et AX-12 (Protocol 1.0) sur le même bus RS-485. Elle utilise le mécanisme **GroupSyncWrite** pour envoyer une position à plusieurs moteurs en un seul paquet série, réduisant ainsi la latence.

```
% Instanciation
hw = hardware.DynamixelInterface(params);
hw.init();

% Écrire une position (rad) sur un moteur
hw.writePosition(motor_id, angle_rad);

% Lire la position actuelle
q_actual = hw.readPosition(motor_id);

% Écriture synchrone sur toutes les pattes
hw.writeAllLegsSync(angles_matrix);

% Nettoyage (désactivation couple + fermeture port)
hw.cleanup();
```

Méthode	Description
init()	Ouvre le port COM, configure le baud rate, active le couple, initialise GroupSyncWrite
writePosition(id, q)	Convertit rad → unités Dynamixel, applique offset/direction, envoie la commande
readPosition(id)	Lit la position brute, convertit en rad (modèle)
writeAllLegsSync(Q)	Diffuse les 8 positions en un seul paquet SyncWrite
holdZAxis()	Maintient les moteurs AX-12 à une position fixe avec compliance
cleanup()	Désactive le couple sur tous les moteurs, ferme le port

Transformation moteur ↔ modèle : Les pattes gauches et droites sont montées en miroir. Les paramètres DIRECTIONS et OFFSETS dans `robot_params.m` corrigent ce décalage sans duplication de code.

Registre	XM430 (adresse)	AX-12 (adresse)	Fonction
TORQUE_ENABLE	64	24	Activation du couple moteur
GOAL_POSITION	116	30	Position cible (0–4095)
PRESENT_POSITION	132	36	Position mesurée (0–4095)
PROFILE_VELOCITY	112	—	Vitesse de profil (XM430)
MOVING_SPEED	—	32	Vitesse cible (AX-12)

5.2 DynamixelInterfaceVMC.m — Contrôle en Couple

Cette variante est destinée au contrôle VMC. Elle bascule les moteurs en **mode courant (mode 0)** et expose la méthode `writeTorque(id, tau)` qui convertit un couple en N·m en unité brute de courant (constante : 208.85 unités/N·m).

```
% Conversion couple
%  $\tau$  (N·m) → courant_brut
%  $K_{\text{torque}} \approx 1.78 \text{ N·m/A}$  | résolution : 2.69 mA/unité
raw = tau * 208.85;
```

5.3 Utilitaires de Conversion — rad2dxi / dxi2rad

```
% Radians  $[0, 2\pi]$  → unités Dynamixel  $[0, 4095]$ 
dxi_val = utils.rad2dxi(angle_rad);

% Unités Dynamixel  $[0, 4095]$  → radians  $[0, 2\pi]$ 
angle_rad = utils.dxi2rad(dxi_val);
```

La conversion est linéaire : 4096 pas par révolution (résolution 0,088°/pas). La fonction `dxi2rad` effectue un cast en double pour éviter les corruptions d'arithmétique entière non signée sous MATLAB.

6 Scripts Principaux d'Exécution

Tous les scripts principaux suivent le même patron d'exécution :

1. Initialiser les paramètres (`robot_params`)
2. Créer les interfaces matérielles (`DynamixelInterface`)
3. Valider la trajectoire dans l'espace de travail (`calc_workspace`)
4. Lever le robot en douceur (`ramp 3 secondes, smoothstep`)
5. Boucle temps réel (`tic/toc, pause pour respecter Ts`)
6. Nettoyage sécurisé (`hw.cleanup` dans `try/catch`)

6.1 Démarche Quadrupède Standard — `main_hardware_standard_gait.m`

Contrôle des quatre pattes simultanément en démarche trot diagonale. La trajectoire de chaque pied est une **cycloïde** (phase d'oscillation) suivie d'un glissement linéaire (phase d'appui).

Paramètre	Valeur	Description
Fréquence	2.0 Hz	Fréquence de démarche
Ts	0.01 s	Période d'échantillonnage
Xc	9 mm	Centre X de la trajectoire
Yc	-150 mm	Hauteur de démarche
A	40 mm	Demi-amplitude X
B	20 mm	Hauteur de levée du pied
Phase FR	0.0	Patte avant-droite
Phase FL	0.5	Patte avant-gauche
Phase RR	0.5	Patte arrière-droite
Phase RL	0.0	Patte arrière-gauche

Commandes clavier durant l'exécution :

- **ESPACE** : basculer entre marche et position debout.
- **q** : arrêter proprement (`cleanup` automatique).

6.2 Trajectoire Cycloïde — `main_hardware_cycloid.m`

Test sur une seule patte avec une trajectoire cycloïde. Le script enregistre les positions de référence et réelles, puis trace les erreurs de suivi après l'exécution.

```
% Configurer la patte à tester
LEG = 'RIGHT'; % ou 'LEFT'

% Paramètres cycloïde
Xc = 5;    Yc = -160; % Centre de trajectoire (mm)
A  = 50;   B  = 20;  % Amplitude (mm)
freq = 1;          % Fréquence (Hz)
```

6.3 Trajectoire Elliptique — main_hardware_ellipse.m

Similaire au script cycloïde, mais utilise une trajectoire elliptique. Utile pour comparer la douceur de mouvement et les erreurs de suivi entre les deux formes de trajectoire.

```
Xc = 40;  Yc = -145; % Centre (mm)
A  = 20;  B  = 15;  % Demi-axes (mm)
freq = 0.5;          % Fréquence (Hz)
```

6.4 Démarche Lente — main_hardware_slow_gait.m

Variante de la démarche quadrupède avec phases décalées différentes (gait walk) et fréquence réduite (2.2 Hz). Cette configuration est plus stable à basse vitesse.

Paramètre	Valeur	Vs. standard
Fréquence	2.2 Hz	2.0 Hz
Phase FL	0.75	0.50
Phase RR	0.50	0.50
Phase RL	0.25	0.00
Xc	25 mm	9 mm
A	30 mm	40 mm
B	15 mm	20 mm

7 Tests et Visualisation

7.1 Visualiseur Interactif — tests/leg_viewer.m

Interface graphique MATLAB permettant d'explorer la cinématique de la patte en temps réel via deux curseurs (sliders) :

- **Slider θ** : angle de hanche (90° – 160°).
- **Slider a** : angle de manivelle (95° – 175°).

La fenêtre affiche tous les points A–H du mécanisme, le nuage de points de l'espace de travail en arrière-plan, et la position courante du pied dans le titre de la figure.

```
% Lancer le visualiseur
cd tests
leg_viewer
```

7.2 Analyse de l'Espace de Travail — tests/test_workspace.m

Procédure de validation complète en quatre étapes :

Étape 1 — Vérification de cohérence $FK \rightarrow IK \rightarrow FK$: Calcule l'erreur maximale de bouclage pour détecter des incohérences de branche.

Étape 2 — Balayage de l'espace de travail : Grille 60×60 sur l'espace articulaire, collecte des points atteignables.

Étape 3 — Visualisation : Nuage de points + contour convexe.

Étape 4 — Validation ponctuelle $IK \rightarrow FK$: 8 cibles aléatoires : compare la position demandée à la position atteinte.

```
cd tests
test_workspace
```

7.3 Trajectoire Biologique — tests/main_cycloid_fennec.m

Ce script rejoue une trajectoire de démarche capturée sur un animal (fennec) dont les points de pied sont stockés dans **fennec_foot_points.csv**. Les coordonnées en pixels sont normalisées vers l'espace de travail du robot. Deux pattes sont synchronisées en opposition de phase (décalage $T/2$).

■ L'utilisateur doit confirmer visuellement la superposition de la trajectoire fennec avec l'espace de travail avant de démarrer la boucle matérielle.

8 Paramètres de Configuration Centraux

L'ensemble des paramètres du robot est centralisé dans `config/robot_params.m`. Ce fichier est le point d'entrée unique pour toute adaptation matérielle ou géométrique.

Paramètre	Valeur par défaut	Unité	Description
PORT_NAME	'COM5'	—	Port série de l'interface U2D2
BAUD_RATE	1 000 000	bps	Débit série (1 Mbaud)
PROTOCOL_VER	2.0	—	Version du protocole Dynamixel (XM430)
IDS_THIGH	[1,3,7,5]	—	IDs moteurs de cuisse (FR,FL,RR,RL)
IDS_CRANK	[2,4,8,6]	—	IDs moteurs de manivelle
IDS_AX12	[12,11,14,13]	—	IDs moteurs AX-12 (axe latéral)
DIRECTIONS	[1,-1,1,-1]	—	Correction symétrie miroir
OFFSETS	[...]	rad	Offsets de zéro par moteur
Ts	0.01	s	Période d'échantillonnage (100 Hz)
L1	40	mm	Longueur bielle AB
L2	100	mm	Longueur cuisse DG
L3	105.73	mm	Longueur tibia GH

Bonne pratique

Ne jamais dupliquer ces valeurs dans d'autres fichiers. Toujours importer la structure `cfg = robot_params()` au début de chaque script et la passer en argument aux modules de cinématique et de contrôle.

9 Bonnes Pratiques et Sécurité

La sécurité est une contrainte critique dans tout système robotique embarqué. CSable intègre plusieurs mécanismes de protection :

Vérification de l'espace de travail	La trajectoire complète est validée contre le nuage de points atteignables avant le démarrage de la boucle matérielle. Toute position hors portée interrompt l'exécution.
Gestion des singularités	La cinématique inverse retourne [NaN ; NaN] dès qu'un point est hors de portée. Tous les contrôleurs maintiennent la dernière action valide dans ce cas.
Limitation de vitesse	Un filtre de taux maximal (240°/s) est appliqué à chaque pas pour éviter les sauts d'angle qui endommagent les articulations.
Saturation du couple (VMC)	Le couple est clampé à ± 3.0 N·m (couple de décrochage moteur : 4.1 N·m) pour préserver les réducteurs.
Levée en douceur	Chaque script utilise une interpolation smoothstep sur 3 secondes pour amener le robot de la position repliée à la position debout.
Try/Catch global	La boucle principale est encapsulée dans un bloc try/catch. En cas d'exception, <code>hw.cleanup()</code> désactive immédiatement le couple sur tous les moteurs.
SyncWrite	L'envoi groupé des positions évite les désynchronisations entre pattes dues à la latence série cumulée des commandes individuelles.

- Ne jamais exécuter un script principal sans vérifier physiquement que le robot est dans une position de départ sûre (pattes à mi-course, espace libre autour du robot). En cas de doute, couper l'alimentation avant de lancer MATLAB.

10 Glossaire

Boucle ouverte	Stratégie de commande sans retour d'état — les sorties ne sont pas mesurées pour corriger l'entrée.
Cinématique directe (FK)	Calcul de la position de l'effecteur (pied) à partir des angles articulaires.
Cinématique inverse (IK)	Calcul des angles articulaires à partir de la position souhaitée de l'effecteur.
Compliance	Souplesse mécanique ou logicielle d'un actionneur face à une force externe.
Dynamixel	Famille de servomoteurs intelligents ROBOTIS avec contrôle de position/vitesse/couple en série.
GroupSyncWrite	Mécanisme du SDK Dynamixel permettant d'envoyer les consignes à plusieurs moteurs en un seul paquet.
Jacobien	Matrice de sensibilité liant les vitesses articulaires aux vitesses de l'effecteur.
MPC	Model Predictive Control — commande par optimisation sur un horizon temporel glissant.
Mécanisme 4 barres	Liaison mécanique à quatre segments rigides formant un quadrilatère articulé.
Phase de gait	Fraction normalisée du cycle de démarche à laquelle chaque patte commence son oscillation.
Protocol 2.0	Protocole de communication série de ROBOTIS pour les moteurs de la série X (XM430, etc.).
Singularité	Configuration articulaire où le Jacobien est singulier (rang insuffisant), empêchant la commande.
SyncWrite	Voir GroupSyncWrite.
Trot diagonal	Allure où deux pattes diagonalement opposées se lèvent en même temps.
VMC	Virtual Model Control — commande par impédance utilisant des ressorts/amortisseurs virtuels.
Workspace	Espace de travail atteignable — ensemble des positions accessibles par le pied.

Document généré le 09 June 2026 — CSable v1.0.0 — Département Robotique